VERIQTA

50 REDIS INTERVIEW SCENARIOS:

QUESTIONS, ANSWERS, AND PRODUCTION TROUBLESHOOTING



@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

1. Redis Is Fast, but Your Application Is Timing Out

Scenario 01

Redis reports low CPU utilization, but application requests exceed their latency SLO. What do you investigate before scaling Redis?

Answer and Investigation

1. Check network latency between application and Redis (round trip time).
2. Inspect client-side connection pool and queuing.
3. Look for blocked commands, large values or big keys.
4. Check SLOWLOG for slow commands.
5. Verify memory usage, eviction and swap activity.
6. Check for limitations on file descriptors and connections.
7. Examine client retry and timeout configuration.

Useful Commands

```
redis-cli --latency
redis-cli INFO stats
redis-cli INFO memory
redis-cli CLIENT LIST
redis-cli SLOWLOG GET 10
redis-cli LATENCY DOCTOR
redis-cli CONFIG GET *timeout*
```

Key Observations

- High network latency increases response time even with low CPU.
- Client-side queueing can cause timeouts.
- Large keys or slow commands block the event loop.
- Memory swap and eviction increase latency.
- Too many connections reduce performance.

Scenario 02

Redis latency increases every few minutes, yet command throughput remains stable. How do you distinguish persistence-related stalls from network latency?

Answer and Investigation

1. Check persistence activity (RDB snapshot or AOF rewrite).
2. Correlate latency spikes with background save events.
3. Use LATENCY DOCTOR to identify internal events.
4. Compare server-side latency with client-side network latency.
5. Check disk I/O, filesystem latency and available disk space.
6. Verify if latency occurs only on write-heavy workloads.

Useful Commands

```
redis-cli LATENCY DOCTOR
redis-cli INFO persistence
redis-cli INFO stats
redis-cli SLOWLOG GET 10
redis-cli INFO cpu
iostat -x 1     # on Redis server
df -h           # check disk space
```

What to Look For

- RDB or AOF rewrite events matching the latency spikes.
- High disk I/O or filesystem latency.
- Latency events like fork, aof-write, rdb-save in LATENCY DOCTOR.
- If network latency, spikes will appear between client and server, not in Redis internal latency.

Scenario 03

A single Redis command delays thousands of unrelated requests. How can this happen even when the server has multiple I/O threads?

Answer and Explanation

1. Redis executes commands in a single-threaded event loop.
2. I/O threads only handle network read/write, not command execution.
3. A long-running command (e.g. large KEYS, big SORT, blocking Lua script) blocks the event loop.
4. While the command runs, all other commands wait, even from different clients.
5. This causes thousands of requests to queue and experience high latency.

Examples of Blocking Commands

- KEYS * (on large keyspace)
- SORT on large dataset
- Complex Lua script
- Big SMEMBERS / LRANGE
- Large JSON operations
- Massive DEL of millions of keys
- BGSAVE fork with high memory (copy-on-write impact)

How to Diagnose

- redis-cli SLOWLOG GET 10
- redis-cli LATENCY DOCTOR
- Monitor command durations
- Check for long-running scripts
- Look at command arguments and key sizes
- Correlate with timestamp of latency spike

Production Focus

Command execution | Event-loop blocking | SLOWLOG | LATENCY DOCTOR | Network round trips | Persistence activity | Latency attribution

VERIQTA

2. The Redis Commands That Quietly Destroy Performance

Scenario 04

An engineer replaces KEYS with SCAN, but production latency still increases. Why is the change not automatically safe?

Answer and Explanation

1. SCAN is non-blocking, but it is still expensive if you use a large COUNT, run multiple scans in parallel, or process big keys.
2. SCAN iterates the entire keyspace and can still consume CPU, memory and network bandwidth.
3. High COUNT, fast iteration, or heavy client-side processing can overwhelm Redis and the client.
4. SCAN does not guarantee no duplicates, so clients may reprocess keys, increasing load.

Example Commands

```
# Scan with a safe count
redis-cli  SCAN 0 COUNT 100

# Iterate in a loop
while true; do
   redis-cli  SCAN $cursor
   COUNT 100
   # process keys
   # update cursor
done
```

Key Takeaways

- Use small COUNT (e.g. 100–500).
- Add delay between iterations.
- Avoid running many SCAN operations in parallel.
- Do not perform heavy work on the Redis server.
- Handle duplicate keys.
- Monitor CPU, network bandwidth and client processing time.
- For large maintenance tasks, run during low traffic or use Redis Cluster hash slots.

Scenario 05

A pipeline reduces network round trips but causes memory pressure and long response delays. What went wrong?

Answer and Explanation

1. Pipelining queues many commands and Redis processes them, then returns all responses at once.
2. If the pipeline is too large, the response payload can be huge, consuming server and client memory.
3. Large pipelines increase output buffer memory, which can cause latency spikes and even client disconnections.
4. Pipelining is most effective with moderate batch sizes that match your workload and network conditions.

Example (Bad vs Good)

❌ Too large (can cause memory pressure)

```
pipeline = []
for i in range(100000):
   pipeline.append("GET key"+str(i))
  redis_client.execute(pipeline)
```

✅ Use a reasonable batch size

```
batch = 500  # or 100–1000
for chunk in chunks(keys, batch):
   pipeline = ["GET " + k for k in chunk]
   redis_client.execute(pipeline)
```

What to Monitor

- Client output buffer size (INFO clients).
- Memory usage (INFO memory).
- Network bandwidth.
- Latency (LATENCY DOCTOR).
- Large replies (big values, large lists/sets).
- Adjust pipeline batch size based on value size and network capacity.

Scenario 06

A Lua script executes correctly in staging but blocks production traffic. How would you diagnose and redesign it?

Answer and Explanation

1. Lua scripts run atomically and block the Redis event loop for the entire execution time.
2. A script that processes large datasets, loops over many keys or performs heavy computations can block all other commands.
3. What works in staging with small data can fail in production with real data volumes and higher concurrency.
4. Redesign the script to process smaller batches, use incremental iteration, or move heavy logic to the client.

Diagnostics

```
# Find slow scripts/commands
redis-cli SLOWLOG GET 10
redis-cli LATENCY DOCTOR

# Check running script
redis-cli SCRIPT EXISTS <sha>
redis-cli SCRIPT KILL

# Monitor command latency
redis-cli INFO commandstats
```

Best Practices

- Keep scripts short and bounded (e.g. < 5–50 ms).
- Avoid unbounded loops.
- Process keys in small batches using SCAN.
- Use timeouts and monitoring.
- Consider moving complex logic to the application.
- Test with production-like data volume before deploying.

Production Focus

Incremental iteration | Cursor behavior | Batch sizes | Pipeline response buffering | Script execution limits | Workload-aware command selection

VERIQTA

3. The Hot Key That Defeated Horizontal Scaling

Scenario 07

One Redis Cluster shard reaches saturation while the other shards remain underutilized. How do you prove that a hot key is responsible?

Answer and Investigation

1. Check per-node command and key hot spots.
2. Identify top keys by request count and latency.
3. Verify that the hot key maps to the saturated shard.
4. Compare key distribution across all shards.
5. Confirm with client-side metrics (which endpoint is hit most).
6. Use MONITOR, SLOWLOG and keyspace analysis.
7. Check if the hot key has high read/write frequency or large data size.

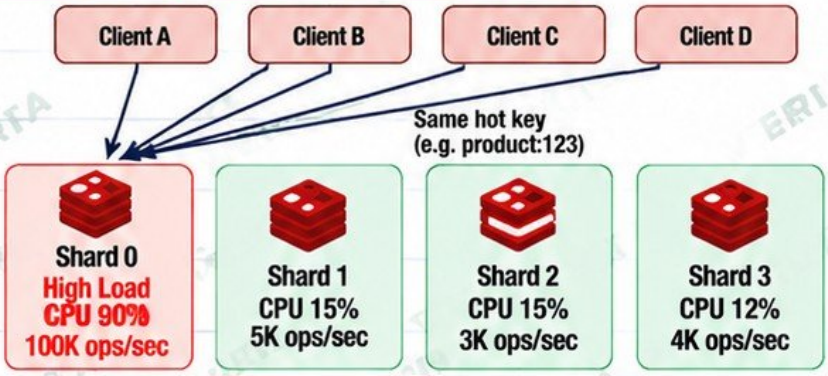
Useful Commands

```
# Check key distribution
redis-cli -c CLUSTER KEYSLOT <key>
# Find hot keys (use redis-cli or redis-top if available)
redis-cli --hotkeys
# Check per-shard stats
redis-cli -c CLUSTER NODES
# See slow commands
redis-cli SLOWLOG GET 10
# Monitor live traffic (use in test)
redis-cli MONITOR
```

Key Evidence

- One shard has much higher ops/sec and CPU.
- SLOWLOG shows the same key repeatedly.
- MONITOR shows the same key being requested frequently.
- CLUSTER KEYSLOT confirms the key belongs to the busy shard.
- Other shards remain underutilized.
- High network traffic to a single shard endpoint.

How a Hot Key Looks in a Redis Cluster



How to Fix / Mitigate

1. Use key sharding (e.g. product:123:0..N).
2. Add local caching for very hot keys.
3. Use read replicas for read-heavy keys.
4. Cache at application layer with TTL and jitter.
5. Consider request coalescing (single flight).
6. Use consistent hashing across multiple keys.
7. For large values, split into smaller keys.
8. Use a CDN or edge cache for global hot keys.
9. Monitor and alert on key-level QPS and latency.

Scenario 08

A frequently requested key causes repeated backend failures immediately after expiration. Why can adding more Redis nodes fail to solve this?

Answer and Explanation

1. When the key expires, many clients simultaneously request it (cache stampede).
2. All requests go to the backend to rebuild the value.
3. This overloads the backend, causing failures or timeouts.
4. Adding more Redis nodes does not solve it because the problem is the thundering herd, not Redis capacity.
5. The key is missing, so every client, regardless of which node they hit, still triggers a backend request.

How to Prevent Cache Stampede

- Use TTL with random jitter.
- Use request coalescing (singleflight).
- Use a short fallback value (stale-while-revalidate).
- Pre-warm critical keys before expiration.
- Use distributed locks (SET NX) to allow only one client to rebuild.
- Increase TTL for expensive keys.
- Use background refresh to refresh keys early.
- Use local caching to reduce simultaneous misses.

Example Pattern (Singleflight)

```
# Acquire lock to rebuild key
SET lock:product:123 1
NX EX 30

# Only one client rebuilds, others wait
if lock acquired:
    fetch from backend
    set key with new TTL
    DEL lock:product:123
else:
    wait and retry after short delay
```

Production Focus

Hot-key detection | Request distribution | Cache stampedes | Key replication strategies | Local caching | Request coalescing | Backend protection

VERIQTA

4. Redis Has Free Memory, but Writes Are Failing

Scenario 09

Redis rejects writes with an OOM error even though the host reports available memory. What explains the discrepancy?

Answer and Explanation

1. Redis enforces maxmemory and rejects writes when its own memory limit is reached, regardless of host free memory.
2. Some memory is not evictable (e.g. replication buffers, AOF buffers, internal structures).
3. Memory fragmentation and allocator overhead can make used_memory_rss much higher than data size.
4. Even with free host memory, Redis will still refuse writes if memory policy and limits are reached.

Useful Commands

```
redis-cli INFO memory
redis-cli CONFIG GET maxmemory
redis-cli CONFIG GET maxmemory-policy
redis-cli INFO stats | grep evicted
redis-cli INFO replication
redis-cli MEMORY STATS
redis-cli CLIENT LIST
redis-cli INFO persistence
```

Key Points to Check

- maxmemory value and policy.
- used_memory vs used_memory_rss.
- mem_fragmentation_ratio.
- evicted_keys count.
- replication backlog size.
- AOF buffer size.
- number of clients and output buffers.
- large keys or non-evictable data.
- verify which memory categories are consuming memory.

Scenario 10

Redis memory usage appears stable, but the operating system terminates the process during persistence. What happened?

Answer and Explanation

1. During RDB save or AOF rewrite, Redis forks a child process, and the OS uses copy-on-write (CoW).
2. The forked process needs additional memory for page tables and any modified pages.
3. If memory is tight, the OS OOM killer may terminate Redis even though main memory usage looked stable.
4. High write traffic during persistence increases CoW memory, making the spike larger than expected.

Evidence and Commands

```
redis-cli INFO memory
redis-cli INFO persistence
redis-cli INFO stats
ps aux | grep redis
dmesg | grep -i oom
cat /var/log/kern.log | grep -i oom
redis-cli CONFIG GET save
redis-cli CONFIG GET appendonly
```

Key Points to Investigate

- RDB or AOF rewrite in progress.
- available host memory at time of fork.
- write traffic rate during persistence.
- kernel OOM killer logs.
- used_memory_rss and mem_fragmentation_ratio.
- swap configuration.
- large dataset with high churn.
- container memory limits (if in Docker/ Kubernetes).

Scenario 11

Eviction is enabled, but Redis still experiences memory-related failures. Which memory categories and policies would you inspect?

Answer and Explanation

1. Eviction only applies to keys eligible for eviction based on the configured policy (allkeys or volatile).
2. If most keys have no TTL and the policy is volatile-*, no keys will be evicted.
3. Non-evictable memory (replication buffers, AOF buffers, client output buffers, Lua scripts, internal structures) can still exhaust memory.
4. Large values, high fragmentation, or slow eviction rate can also cause memory pressure and failures.

Important Commands

```
redis-cli CONFIG GET maxmemory
redis-cli CONFIG GET maxmemory-policy
redis-cli INFO memory
redis-cli INFO stats | grep evicted
redis-cli INFO replication
redis-cli CLIENT LIST
redis-cli MEMORY USAGE <key>
redis-cli INFO commandstats
```

Memory Categories to Check

- Key data size (MEMORY USAGE).
- TTL coverage and key distribution.
- evicted_keys and keyspace hits/misses.
- replication backlog and replica buffers.
- AOF buffers and rewrite activity.
- client output buffers (slow clients).
- Lua scripts and module memory.
- memory fragmentation ratio.
- allkeys-* vs volatile-* policy.
- large keys or high-cardinality data.

Production Focus

maxmemory and policy | Eviction eligibility | Memory fragmentation | Replication and AOF buffers | Fork and CoW overhead | Host memory limits

VERIQTA

5. Your Cache Eviction Policy Is Silently Breaking the Application

Scenario 12

Cache hit rate collapses after switching to an LRU-based eviction policy. Why might the new policy perform worse?

Answer and Explanation

1. Redis uses Approximate LRU, not exact LRU. It samples keys, so frequently used keys can still be evicted.
2. If you switched from a policy like *allkeys-lfu* or *volatile-lfu* to an LRU-based policy, long-lived but frequently used keys may now be evicted.
3. If many keys have similar access patterns, LRU cannot distinguish "important" hot keys from one-time keys.
4. Sudden traffic changes can cause recently unused but important keys to be evicted.
5. LRU works better for workloads with clear temporal locality, but performs poorly for skewed access patterns.

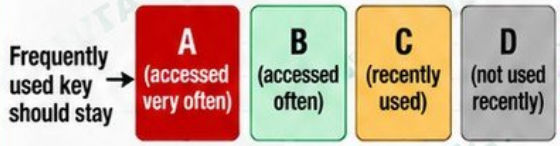
Useful Commands

```
redis-cli INFO stats
redis-cli INFO memory
redis-cli INFO keyspace
redis-cli CONFIG GET maxmemory
redis-cli CONFIG GET maxmemory-policy
redis-cli INFO commandstats
redis-cli MONITOR
redis-cli LATENCY DOCTOR
redis-cli SLOWLOG GET 10
```

Key Things to Check

- Current maxmemory-policy.
- Cache hit rate before and after policy change.
- evicted_keys and keyspace_hits from INFO stats.
- Key access patterns (hot vs cold).
- TTL usage and key distribution.
- Application queries (are many unique keys being generated?).
- Whether hot keys have TTL and are included in the policy.

How LRU Can Perform Worse



Approximate LRU (sampling)



Redis samples a few keys and may not see the real usage pattern, so A can be evicted even if it is frequently used.

Better Options for Different Workloads

- Use LFU (allkeys-lfu) for skewed workloads with clear hot keys.
- Use longer TTL for important keys.
- Avoid generating too many unique keys.
- Monitor key usage before and after policy changes.

Scenario 13

A Redis instance contains millions of keys, but the expected keys are not being evicted. How do TTL coverage and eviction-policy selection affect the result?

Answer and Explanation

1. Eviction only applies to keys that are eligible based on the configured policy.
2. If you use a volatile-* policy, only keys with an expiration (TTL) can be evicted. Keys without TTL will never be evicted.
3. If most keys do not have TTL, memory usage can grow until writes fail, even with eviction enabled.
4. Check TTL coverage, key distribution and memory usage by key type.
5. Use allkeys-* policies if you want every key to be eligible for eviction, regardless of TTL.

Useful Commands

```
redis-cli CONFIG GET maxmemory
redis-cli CONFIG GET maxmemory-policy
redis-cli INFO memory
redis-cli INFO keyspace
redis-cli INFO stats
redis-cli --scan --pattern "*" | xargs -L 1 \
    redis-cli TTL
redis-cli MEMORY USAGE <key>
redis-cli OBJECT IDLETIME <key>
redis-cli INFO evicted_keys
```

Key Things to Investigate

- What percentage of keys have TTL?
- Are you using volatile-* instead of allkeys-*?
- Are large non-expiring keys consuming most of the memory?
- Check key types (strings, hashes, lists, sets) and their sizes.
- Verify eviction activity (evicted_keys).
- Look for memory fragmentation.
- Confirm application is setting TTL as expected.
- Check for keys that should expire but keep being recreated.

Production Focus

Approximate LRU | LFU vs LRU | allkeys vs volatile policies | Expiration metadata | Workload access patterns | Eviction telemetry

VERIQTA

6. The Redis Key That Expired but Would Not Disappear

Scenario 14

A key disappears from application reads, but memory is not reclaimed immediately. How do Redis expiration mechanisms explain this?

Answer and Explanation

1. Redis uses two expiration mechanisms: passive expiration and active expiration.
2. When a key is accessed, Redis checks if it has expired (passive expiration) and removes it logically.
3. If the key is not accessed, it relies on a background active expiration cycle to find and delete expired keys in batches.
4. Memory is not reclaimed immediately because expired keys are deleted asynchronously.
5. Large values may be freed in the background using lazy freeing, so memory usage appears high for a while.
6. Tools like INFO memory, MEMORY USAGE and expired_keys help you verify the behavior.

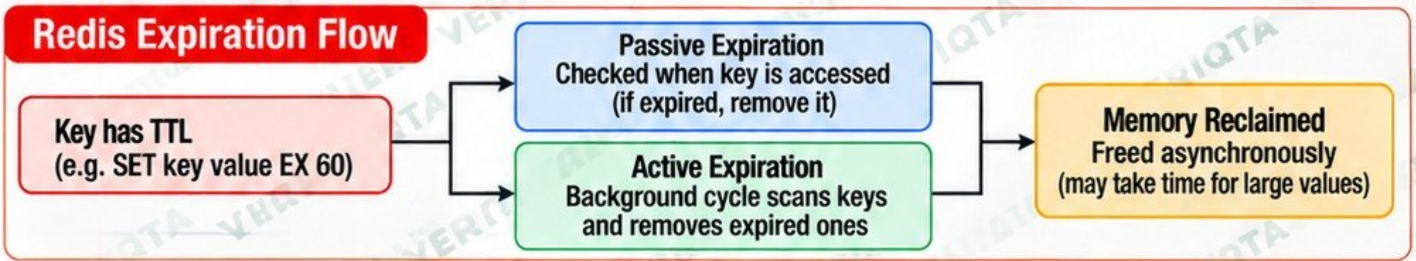
Useful Commands

```
redis-cli GET key
# returns nil if expired
redis-cli TTL key
# check remaining TTL
(-2 = expired, -1 = no TTL)
redis-cli EXISTS key
redis-cli INFO stats | grep expired
redis-cli INFO memory
redis-cli MEMORY USAGE key
redis-cli CONFIG GET lazyfree-lazy-expire
redis-cli CONFIG GET hz
# controls active expiration cycle
redis-cli LATENCY DOCTOR
```

Key Points to Check

- Is the key expired (TTL = -2)?
- Are you relying only on active expiration?
- Check expired_keys in INFO stats.
- Verify lazyfree-lazy-expire setting.
- Look at memory fragmentation and allocator behavior.
- Large values may take time to free even after deletion.
- Memory may not drop immediately due to asynchronous freeing.

Redis Expiration Flow



Scenario 15

A bulk cleanup operation causes unexpected latency and replication pressure. How would you redesign the deletion strategy?

Answer and Explanation

1. Deleting millions of keys at once can block the event loop, increase CPU usage and cause replication lag.
2. DEL is synchronous and can block Redis when deleting large keys or many keys.
3. UNLINK removes keys asynchronously, freeing memory in the background without blocking the main thread.
4. Use SCAN to iterate keys in small batches instead of KEYS.
5. Throttle deletions with batch size and sleep intervals.
6. For predictable cleanup, design key lifecycle with TTLs, key prefixes and background cleanup jobs.

Example Commands

```
# Find keys with a pattern (incremental)
redis-cli --scan --pattern "app:*" \
  | xargs -L 100 redis-cli UNLINK
# Delete in controlled batches
for key in $(redis-cli --scan --pattern
"log:*"); do
  redis-cli UNLINK $key
  sleep 0.1
done
# Check replication lag
redis-cli INFO replication
# Monitor latency during cleanup
redis-cli LATENCY DOCTOR
```

Best Practices

- Use UNLINK instead of DEL for large keys or bulk deletions.
- Use SCAN instead of KEYS.
- Delete in small batches (e.g. 100–1000 keys).
- Add sleep intervals to reduce load on the server and replicas.
- Perform cleanup during low traffic.
- Use TTLs to avoid manual cleanup where possible.
- Monitor replication lag and latency while deleting.
- Consider deleting by key prefix, not individual keys.

Production Focus

Passive vs active expiration | UNLINK vs DEL | Asynchronous freeing | Expiration storms | Replication pressure | Key lifecycle design | Memory reclamation

VERIQTA

7. Redis Restarted Successfully, but Recent Data Vanished

Scenario 16

Redis restarts after a crash and loses recently acknowledged writes. How can this happen with AOF enabled?

Answer and Explanation

1. AOF does not guarantee durability unless appendfsync is set to always.
2. With appendfsync everysec (default), writes can be acknowledged but still be in the OS buffer and not yet flushed to disk.
3. A crash (power loss, kernel panic, OOM kill) can cause data in OS buffers to be lost.
4. Also check if no-appendfsync-on-rewrite yes is enabled, which increases the window of data loss during AOF rewrite.
5. Acknowledged write means accepted by Redis, not necessarily persisted to disk.

Relevant Commands

```
redis-cli CONFIG GET appendfsync
redis-cli CONFIG GET
          no-appendfsync-on-rewrite
redis-cli INFO persistence
redis-cli INFO stats
redis-cli LASTSAVE
redis-cli SLOWLOG GET 10
redis-cli CLIENT LIST
```

Key Takeaways

- appendfsync always = safest (but lower performance).
- appendfsync everysec = up to 1 second of data loss possible.
- appendfsync no = highest risk.
- no-appendfsync-on-rewrite can increase data loss window.
- Use replication and backups for additional protection.
- Test crash recovery in a non-production environment regularly.

Scenario 17

Redis has both RDB and AOF configured, but recovery behavior differs from what the operations team expected. What must be verified?

Answer and Explanation

1. When both RDB and AOF are enabled, Redis gives priority to AOF for recovery if a valid AOF exists.
2. If AOF is corrupted or missing, Redis falls back to the latest RDB snapshot.
3. Verify the actual files present, their timestamps, and if the AOF is complete and consistent.
4. Check for multi-part AOF format (AOF base file and incremental files).
5. Ensure the server was started with the expected configuration and that the AOF was not disabled or truncated.

Commands to Verify

```
redis-cli CONFIG GET save
redis-cli CONFIG GET appendonly
redis-cli INFO persistence
ls -lh /var/lib/redis/  # check files
redis-check-aof --file appendonly.aof
redis-check-rdb --file dump.rdb
redis-cli CONFIG GET dir
```

Recovery Priority

1. Use AOF if valid (appendonly.aof)
2. If AOF is missing or corrupted
3. Fallback to latest RDB snapshot (dump.rdb)

Scenario 18

AOF rewrite activity coincides with disk pressure and degraded performance. How do you investigate without risking further data loss?

Answer and Explanation

1. AOF rewrite (BGREWRITEAOF) creates a child process and new AOF file. This increases disk I/O and memory (copy-on-write).
2. Disk pressure can slow down the rewrite, causing increased latency and possible write stalls.
3. Check disk usage, AOF size, rewrite status and I/O wait without killing the process.
4. Do not manually delete or truncate AOF files during rewrite as this can cause data loss.
5. Consider delaying new rewrites, freeing disk space, or moving AOF to a dedicated disk.

Investigation Commands

```
redis-cli INFO persistence
redis-cli INFO memory
redis-cli INFO stats
redis-cli INFO cpu
redis-cli LATENCY DOCTOR
redis-cli BGREWRITEAOF
redis-cli CONFIG GET auto-aof-rewrite-percentage
redis-cli CONFIG GET auto-aof-rewrite-min-size
df -h       # check disk space
iostat -x 1   # check disk I/O
redis-cli SLOWLOG GET 10
```

Mitigation Options

- Free disk space (remove old logs, unused files).
- Move AOF to a larger or dedicated disk/volume.
- Temporarily disable auto rewrite if necessary (with caution).
- Adjust auto-aof-rewrite-percentage and auto-aof-rewrite-min-size.
- Monitor latency, CPU, memory and disk I/O during rewrite.
- Ensure you have recent backups before making changes.

Production Focus

appendfsync settings | AOF durability windows | RDB snapshots | Multi-part AOF manifests | Rewrite behavior | Disk saturation | Recovery testing

VERIQTA

8. Redis Replication Is Healthy, but the Data Is Not Consistent

Scenario 19

The primary acknowledges a write, then fails before its replica receives the update. Why can automatic failover lose that write?

Answer and Explanation

1. Redis replication is asynchronous. A write is acknowledged after being written to the primary's memory, not after it reaches replicas.
2. If the primary fails before replicas receive the write, the data exists only on the failed primary and is lost during failover.
3. Sentinel or Cluster will promote a replica that does not have the latest write.
4. This is expected behavior and is the data-loss window in asynchronous replication.

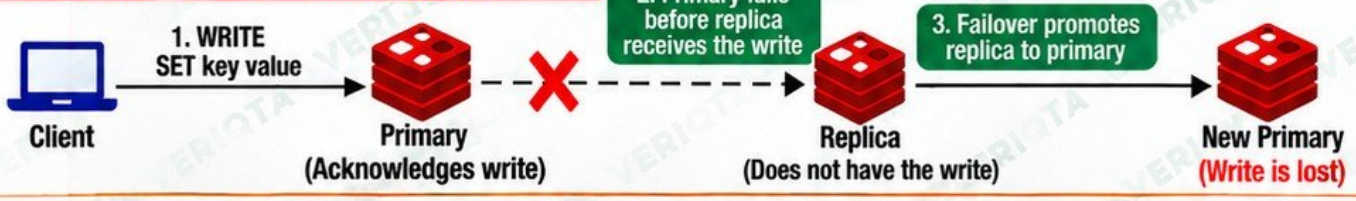
Useful Commands

```
redis-cli INFO replication
redis-cli INFO stats
redis-cli REPLCONF GETACK *
redis-cli WAIT 1 5000
# check replica offset and lag
redis-cli INFO replication
# compare master_repl_offset
# and slave_repl_offset
```

Key Takeaways

- Acknowledged write ≠ durable write.
- Replication is asynchronous.
- Automatic failover can lose recent writes.
- Use WAIT or stronger strategies if the use case requires higher durability.
- For critical data, consider synchronous replication, disk persistence or external systems.

How a Write Can Be Lost During Failover



Scenario 20

Replica lag remains low, but an application occasionally reads stale values. What consistency guarantee is missing?

Answer and Explanation

1. Low replica lag only means the replica is nearly up to date, not that it is always current at read time.
2. Redis provides eventual consistency, not strong consistency, for replica reads.
3. Application may read from a replica before it has received the latest write, especially during failover, network latency or transient lag.
4. There is no read-after-write consistency guarantee unless the application reads from the primary or uses additional mechanisms.

Commands to Investigate

```
redis-cli INFO replication
redis-cli INFO stats
redis-cli REPLCONF GETACK *
redis-cli MONITOR
redis-cli --latency
redis-cli LATENCY DOCTOR
# check replication lag in real time
redis-cli INFO replication | grep lag
```

How to Reduce Stale Reads

- Read from the primary for critical reads after a write.
- Use WAIT to ensure write visibility on replicas (best effort).
- Implement read-after-write logic in the application.
- Use versioning or timestamps.
- Consider sticky reads (read from same node after write).
- Accept eventual consistency where appropriate.

Scenario 21

An engineer uses WAIT and assumes Redis now provides strongly consistent writes. Why is that conclusion unsafe?

Answer and Explanation

1. WAIT ensures the write has been received by the specified number of replicas, but it does not make the write durable.
2. Replicas may acknowledge the write in memory but still lose it if they fail before persisting it to disk.
3. WAIT does not guarantee the write will survive a majority failure, network partition or disk failure.
4. Redis still provides eventual consistency, not strong consistency. WAIT reduces the data-loss window but does not eliminate it.

Example Command

```
redis-cli SET key value
redis-cli WAIT 1 5000
# Wait for 1 replica, 5 seconds
# Returns number of replicas
# that acknowledged the write
```

Key Takeaways

- WAIT ≠ strong consistency.
- WAIT only confirms replicas received the write, not that it is persisted.
- Replicas can still lose acknowledged writes.
- Use AOF with appendfsync, multiple replicas and/or external systems for higher durability.
- Understand the difference between acknowledgment, replication and durability.

Production Focus

Asynchronous replication | Replication offsets | Read-after-write consistency | WAIT behavior | Failover data-loss windows | Acknowledgment vs durability

VERIQTA

9. The Failover That Turned a Minor Outage Into Data Loss

Scenario 22

Redis Sentinel promotes a replica, but the previous primary later returns with additional writes. What happens to those writes?

Answer and Explanation

1. When Sentinel promotes a replica, the old primary becomes a replica and should discard its divergent writes.
2. Any writes accepted by the old primary after failover are not present on the new primary and are lost.
3. Redis does not automatically merge divergent data between the old primary and the new primary.
4. This creates a split-brain window where two nodes accept writes independently.
5. To prevent this, ensure proper fencing, client redirection, and application-level safeguards.

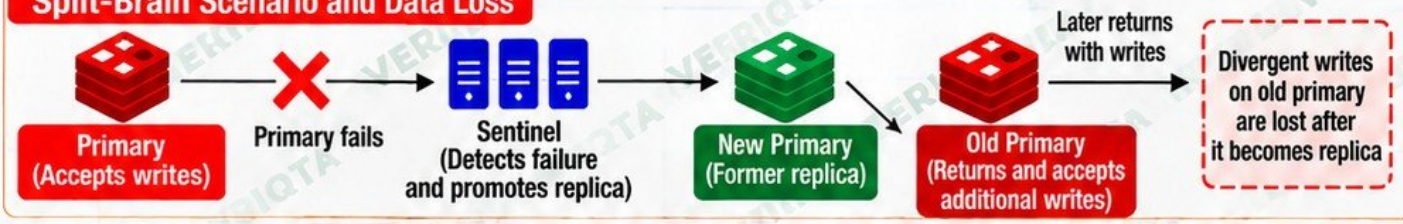
Useful Commands

```
redis-cli SENTINEL get-master-addr-by-name <name>
redis-cli SENTINEL masters
redis-cli SENTINEL replicas <name>
redis-cli INFO replication
redis-cli INFO server
redis-cli CONFIG GET replica-read-only
redis-cli CONFIG GET min-replicas-to-write
redis-cli CONFIG GET min-replicas-max-lag
redis-cli ROLE
redis-cli INFO persistence
```

Key Takeaways

- Writes on the old primary after failover are lost.
- Redis does not perform automatic data merging.
- Implement fencing to stop writes to the old primary.
- Use client libraries that support Sentinel for automatic redirection.
- Consider min-replicas-to-write and min-replicas-max-lag for additional safety.
- Test failover and recovery regularly in a non-production environment.

Split-Brain Scenario and Data Loss



Scenario 23

Sentinel reports the primary as subjectively down, but no failover occurs. How do quorum and authorization requirements explain this?

Answer and Explanation

1. SDOWN means a Sentinel instance thinks the primary is down, but ODOWN requires a quorum of Sentinels to agree.
2. If the number of Sentinels that report SDOWN does not reach the configured quorum, no failover occurs.
3. Sentinel also requires authorization to perform failover. If the failover configuration is incorrect or ACLs block the operation, the failover will not happen.
4. Network partitions or slow Sentinel communication can prevent reaching quorum.
5. Verify Sentinel configuration, quorum, and ACLs when troubleshooting.

Useful Commands

```
redis-cli SENTINEL masters
redis-cli SENTINEL master <name>
redis-cli SENTINEL sentinels <name>
redis-cli SENTINEL get-quorum <name>
redis-cli SENTINEL ckquorum <name>
redis-cli SENTINEL failover <name>
redis-cli SENTINEL auth-pass <name>
redis-cli SENTINEL CONFIG GET *
redis-cli INFO sentinel
```

Key Takeaways

- SDOWN is a local view; ODOWN requires quorum.
- Check the quorum setting in the Sentinel configuration.
- Ensure Sentinel has the required authorization and passwords (ACLs).
- Investigate network issues between Sentinels.
- Confirm that failover is allowed by configuration and not disabled.

Scenario 24

An application continues sending writes to an obsolete primary endpoint after failover. How would you identify and prevent the client-side failure?

Answer and Explanation

1. The application may be using a static primary endpoint instead of Sentinel or a Redis Cluster-aware client.
2. After failover, the old primary becomes a replica and rejects writes (READONLY error) or accepts writes if misconfigured.
3. Check application connection logic, DNS caching, connection pooling, and client library behavior.
4. Use Sentinel-aware clients or implement reconnection and endpoint discovery.
5. Monitor application errors and Redis logs to identify clients writing to the wrong node.

Commands to Investigate

```
redis-cli ROLE
redis-cli INFO replication
redis-cli CLIENT LIST
redis-cli MONITOR
redis-cli CONFIG GET replica-read-only
redis-cli CONFIG GET masterauth
redis-cli CONFIG GET requirepass
redis-cli SENTINEL get-master-addr-by-name <name>
```

Prevention Strategies

- Use Sentinel-aware client libraries with automatic failover handling.
- Avoid hard-coded primary endpoints.
- Implement retry logic and reconnection.
- Use short DNS TTL if using DNS-based discovery.
- Monitor for READONLY errors in the application.
- Test failover scenarios regularly.

Production Focus

SDOWN vs ODOWN | Quorum and leader leection | Failover authorization | Client discovery and redirection | Split-brain windows | Replication reconciliation

VERIQTA

10. The Redis Cluster Is Healthy, but Half the Requests Fail

Scenario 25

Redis Cluster reports healthy nodes, but clients receive repeated MOVED and ASK responses. What would you investigate?

Answer and Explanation

1. MOVED means the client is sending the request to the wrong node for the key's hash slot.
2. ASK means the slot is being migrated and the client should temporarily redirect the request.
3. Investigate client topology cache, whether the client is cluster-aware and updating its view.
4. Check cluster slot ownership and whether resharding is in progress.
5. Verify network issues, stale DNS, load balancers or proxies that may route to the wrong node.

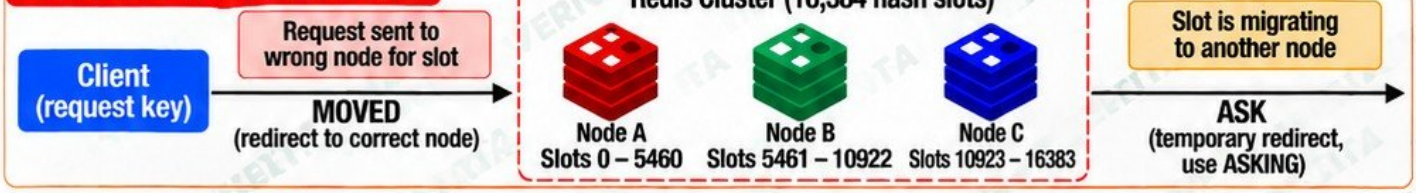
Useful Commands

```
redis-cli -c CLUSTER NODES
redis-cli -c CLUSTER SLOTS
redis-cli -c CLUSTER INFO
redis-cli -c CLUSTER KEYSLOT <key>
redis-cli -c CLUSTER GETKEYSINSLOT <slot> <count>
redis-cli -c CLUSTER COUNTKEYSINSLOT <slot>
redis-cli -c CLUSTER SHARDS
redis-cli -c CLUSTER SAVECONFIG
```

Key Takeaways

- MOVED = wrong node, update topology.
- ASK = slot migrating, send ASKING before the request.
- Use a cluster-aware client with automatic redirection.
- Keep client topology cache updated.
- Check for ongoing resharding or slot migration.
- Avoid misconfigured load balancers or non-cluster-aware proxies.
- Monitor for frequent MOVED/ASK responses.

How MOVED and ASK Work



Scenario 26

A multi-key operation succeeds in development but fails with CROSSSLOT in production. How should the key model change?

Answer and Explanation

1. In Redis Cluster, all keys in a multi-key command must be in the same hash slot.
2. CROSSSLOT occurs when keys hash to different slots (common in multi-key operations like MGET, MSET, SUNION, EVAL, etc.).
3. Use hash tags to ensure related keys are stored in the same slot, e.g. user:{123}:profile and user:{123}:orders.
4. Redesign the key model so related data uses the same hash tag.
5. Consider application-level aggregation if cross-slot operations are unavoidable.

Example

Without hash tag (different slots)

user:123:profile → slot 7421
user:123:orders → slot 9912

✗ CROSSSLOT error

With hash tag (same slot)

user:{123}:profile → slot 4821
user:{123}:orders → slot 4821

✓ Works (same slot)

Useful Commands

```
redis-cli -c CLUSTER KEYSLOT <key>
redis-cli -c CLUSTER GETKEYSINSLOT <slot> <count>
redis-cli -c CLUSTER SLOTS
redis-cli -c CROSSSLOT error (observe in client logs)
```

Key Takeaways

- Use hash tags { } to place related keys in the same slot.
- All keys in a multi-key command must hash to the same slot.
- Redesign the key model for cluster compatibility.
- Use application-side logic if needed.

Scenario 27

A resharding operation causes intermittent errors and increased latency. How do you separate expected migration behavior from client incompatibility?

Answer and Explanation

1. During resharding, slots move between nodes, which can cause ASK responses, temporary redirects and higher latency.
2. Transient errors are expected (ASK, MOVED, TRYAGAIN).
3. If errors persist or clients do not follow redirections, the client may not be cluster-aware or may have a stale topology cache.
4. Check resharding progress and slot states to confirm expected migration behavior.
5. Update or configure the client to properly handle cluster redirections and retry logic.

Commands to Investigate

```
redis-cli -c CLUSTER NODES
redis-cli -c CLUSTER SLOTS
redis-cli -c CLUSTER INFO
redis-cli -c CLUSTER GETKEYSINSLOT <slot> <count>
redis-cli -c CLUSTER COUNTKEYSINSLOT <slot>
redis-cli -c LATENCY DOCTOR
redis-cli -c INFO stats
```

Key Takeaways

- Increased latency and ASK responses are normal during resharding.
- Persistent errors indicate a client compatibility issue.
- Use a cluster-aware client with automatic redirection and retries.
- Monitor slot migration progress.
- Avoid custom or outdated clients that do not support cluster mode.
- Verify no network or DNS issues causing stale routing.

Production Focus

16,384 hash slots | Slot ownership and migration states | Client topology caches | Hash tags and key design | Cluster-aware clients | Error handling and latency impact

VERIQTA

11. A Network Partition Creates Two Versions of Reality

Scenario 28

A Redis primary becomes isolated from the majority of cluster primaries. Why can it temporarily accept writes before eventually refusing them?

Answer and Explanation

1. When a primary is isolated, it does not immediately know that it has lost the majority.
2. It continues to accept writes until it detects that it cannot communicate with a majority of master nodes within the cluster-node-timeout period.
3. After the timeout, the node marks itself as failing and enters a state where it refuses writes to prevent split-brain and divergent data.
4. This delayed behavior allows short network hiccups to recover without unnecessary write unavailability.
5. Once the majority cannot be reached, the node protects data consistency by refusing further writes with a CLUSTER DOWN error.

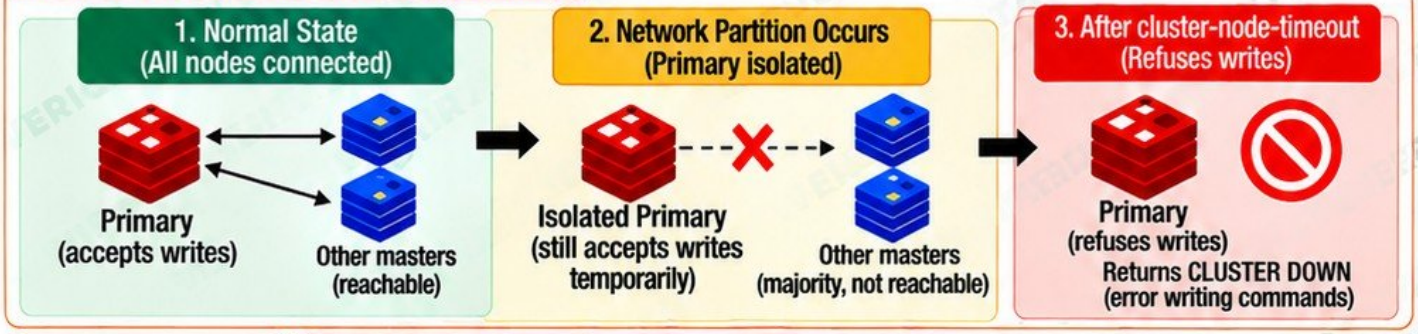
Useful Commands

```
redis-cli -c CLUSTER NODES
redis-cli -c CLUSTER INFO
redis-cli -c CLUSTER MYID
redis-cli -c INFO replication
redis-cli -c CONFIG GET
          cluster-node-timeout
redis-cli -c CONFIG GET
          cluster-require-full-coverage
redis-cli -c INFO stats
redis-cli -c CLUSTER REPLICAS
          <node-id>
```

Key Takeaways

- A primary can accept writes temporarily because it has not yet detected the loss of the majority.
- cluster-node-timeout controls how quickly the node detects the partition.
- After the timeout, the node refuses writes to avoid split-brain and data divergence.
- This behavior balances availability and consistency.
- Monitor cluster state and network stability to minimize unnecessary write failures.

Network Partition Behavior (Timeline)



Scenario 29

A Redis Cluster partition causes some operations to fail even though several nodes remain reachable. How do quorum, slot coverage, and failure detection interact?

Answer and Explanation

1. Redis Cluster requires a majority of master nodes for the cluster to be considered healthy.
2. Even if several nodes are reachable, some operations may fail if the cluster loses majority or if not all hash slots are covered.
3. If cluster-require-full-coverage is enabled (default), Redis will return CLUSTERDOWN when not all slots are available.
4. Failure detection is based on cluster-node-timeout and uses PING/PONG between nodes to detect unreachable nodes.
5. This ensures data safety by preventing writes when the cluster cannot guarantee full consistency across slots.

Commands to Investigate

```
redis-cli -c CLUSTER INFO
redis-cli -c CLUSTER NODES
redis-cli -c CLUSTER SLOTS
redis-cli -c INFO cluster
redis-cli -c CONFIG GET
          cluster-require-full-coverage
redis-cli -c CONFIG GET
          cluster-node-timeout
redis-cli -c CLUSTER COUNTKEYSINSLOT
          <slot>
redis-cli -c CLUSTER GETKEYSINSLOT
          <slot> <count>
redis-cli -c CLUSTER SHARDS
```

Key Takeaways

- Cluster requires majority of master nodes to function normally.
- All 16,384 hash slots must be covered (unless cluster-require-full-coverage is disabled).
- If any slots are unassigned, writes will fail with CLUSTERDOWN.
- Failure detection uses cluster-node-timeout to mark nodes as unreachable.
- This design prioritizes consistency, with availability trade-offs during network partitions.

Production Focus

Cluster majority behavior | Node timeout and failure detection | Slot coverage and cluster state | cluster-require-full-coverage | Availability vs consistency trade-offs

VERIQTA

12. The Redis Connection Storm That Took Down the Application

Scenario 30

Redis CPU remains moderate, but thousands of application connections begin failing after a deployment. What would you inspect?

Answer and Explanation

1. Check the number of connected clients using INFO clients and compare it with maxclients.
2. Verify the server's file descriptor limits (uliimit. and Redis maxclients).
3. Inspect application deployment changes that may have increased the number of connections (e.g. no connection pooling, multiple instances).
4. Check TCP connection states (netstat/ss) for connections in SYN, ESTABLISHED or TIME_WAIT.
5. Look for thundering herd behavior where many application instances create new connections simultaneously.

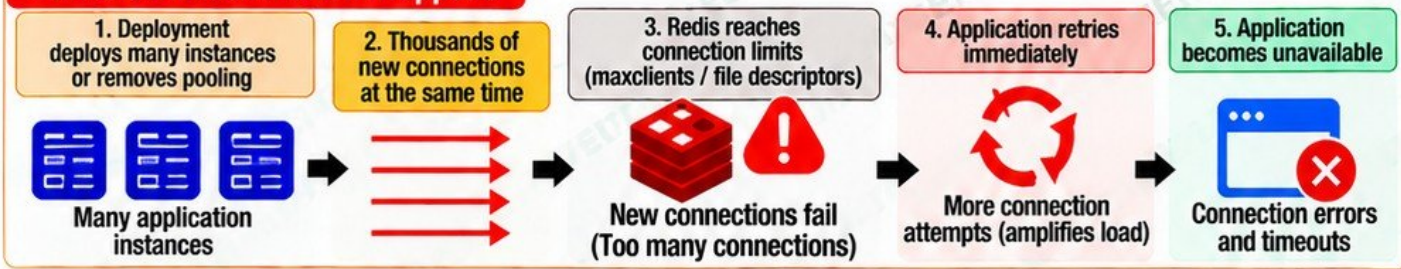
Useful Commands

```
redis-cli  INFO clients
redis-cli  INFO stats
redis-cli  CONFIG GET  maxclients
redis-cli  CONFIG GET  timeout
redis-cli  CLIENT LIST
redis-cli  INFO memory
redis-cli  INFO persistence
# check file descriptor limits
ulimit -n
cat /proc/sys/fs/file-max
ss -ant | grep :6379 | head
netstat -ant | grep :6379 | wc -l
```

Key Takeaways

- A large number of connections can exhaust maxclients or file descriptors even when CPU is low.
- Each connection consumes memory and OS resources.
- Use connection pooling in the application.
- Set appropriate maxclients and ulimit values.
- Monitor connection count and alert on sudden increases.
- Investigate deployment changes that may create new connections per request.

How a Connection Storm Happens



Scenario 31

Redis failover completes, but application recovery takes several minutes. How can connection pools, DNS, and retry policies prolong the outage?

Answer and Explanation

1. Connection pools may keep old connections to the failed primary and take time to refresh.
2. DNS TTL may be high, so clients continue to resolve the old endpoint.
3. Retry policies without backoff can cause repeated failed connections and slow recovery.
4. Some clients cache cluster or Sentinel topology and refresh it infrequently.
5. Applications may need time to detect failures, clear stale connections and establish new ones.

Commands to Check

```
redis-cli  INFO replication
redis-cli  CLIENT LIST
redis-cli  CONFIG GET timeout
redis-cli  CONFIG GET tcp-keepalive
```

Check DNS resolution and TTL

```
nslookup redis.example.com
dig redis.example.com
# check application logs
grep -i redis app.log
```

Key Takeaways

- Use connection pooling with health checks and fast failure detection.
- Set low DNS TTL for Redis endpoints.
- Implement exponential backoff with jitter in retry policies.
- Ensure clients refresh Sentinel or cluster topology quickly.
- Close and recreate stale connections after failover.
- Test failover and recovery in a non-production environment.

Scenario 32

A short network interruption triggers a much larger Redis incident. How do synchronized retries and connection storms amplify the failure?

Answer and Explanation

1. When a network blip occurs, many clients detect a failure at the same time and retry simultaneously.
2. Synchronized retries create a large burst of new connections and commands.
3. This spike can exhaust maxclients, file descriptors, network buffers or CPU, turning a small issue into a major outage.
4. Without jitter, all clients retry in lockstep, making the situation worse.
5. Use exponential backoff with jitter, circuit breakers and connection pooling to prevent retry storms.

Example Retry Strategy

```
# Exponential backoff with jitter (pseudo code)
base = 1  # second
max_wait = 30  # seconds
attempt = 0
while attempt < 5:
    wait = min(max_wait, base * (2 ** attempt))
    wait = wait + random(0, wait * 0.5)  # jitter
    sleep(wait)
    attempt += 1
```

Key Takeaways

- Avoid immediate retries after a failure.
- Use exponential backoff with jitter.
- Implement circuit breakers to stop retry storms.
- Use connection pooling to limit new connections.
- Monitor retry rates and connection metrics.
- Test network failure scenarios regularly.

Production Focus

connected_clients and maxclients | File descriptors and ulimit | TCP connection states | Connection pools and exhaustion | Retry backoff and jitter | Reconnect behavior

VERIQTA

13. Your Redis Distributed Lock Did Not Protect the Resource

Scenario 33

Two workers execute the same critical operation even though both use a Redis lock. What failure modes could allow this?

Answer and Explanation

1. The lock key expires before the first worker completes, allowing a second worker to acquire it.
2. The lock is released incorrectly (no ownership check), allowing another worker to acquire it.
3. Network issues or Redis failover can cause a false lock acquisition in some scenarios.
4. The lock value is not unique per worker, so one worker can accidentally release another worker's lock.
5. Redis lock only coordinates, but the external resource is not protected, so concurrent operations still occur.

Useful Commands

```
# acquire lock with 30 seconds TTL
redis-cli SET lock:resource \\
        "unique-token" NX PX 30000

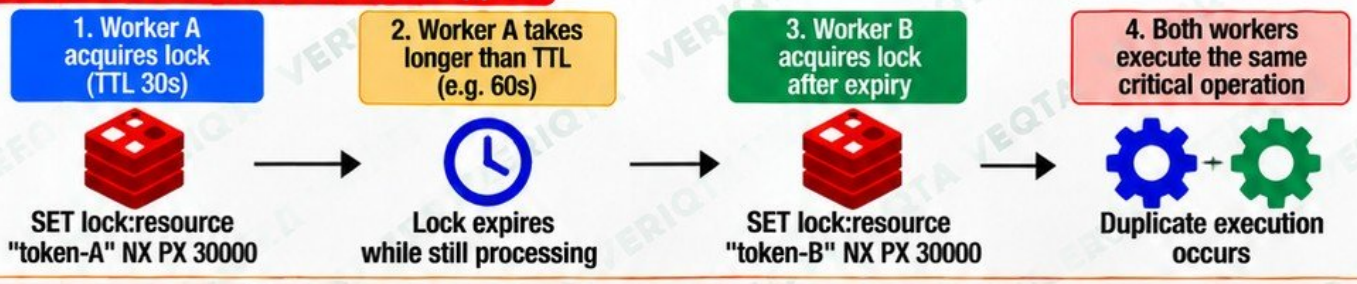
# check lock value
redis-cli GET lock:resource

# release lock (only if value matches)
redis-cli EVAL \\
 "if redis.call('GET', KEYS[1]) ==
 ARGV[1] then return redis.call('DEL',
 KEYS[1]) else return 0 end" \\
 1 lock:resource "unique-token"
```

Key Takeaways

- Always use a unique token for each lock owner.
- Use SET NX PX to acquire locks.
- Release the lock only if you still own it (check the token).
- Handle lock expiration and long processing times.
- A Redis lock coordinates workers but does not automatically protect external systems.
- Consider fencing tokens for critical resources.

How Duplicate Execution Can Happen



Scenario 34

A lock expires while its owner is still processing. Why can a second worker acquire the lock without making the operation safe?

Answer and Explanation

1. Locks in Redis have a TTL (lease). When it expires, the key is removed and another worker can acquire it.
2. The original worker may still be processing the resource, but Redis has no knowledge of the work.
3. This can lead to overlapping operations and data corruption if the external resource is not protected.
4. Use longer TTL, lock renewal (watchdog), or fencing tokens to prevent unsafe concurrent access.

Useful Commands

```
# set a longer TTL
redis-cli SET lock:resource
        "token" NX PX 120000

# check remaining TTL
redis-cli TTL lock:resource

# extend lock (renewal)
redis-cli PEXPIRE lock:resource 30000
```

Key Takeaways

- Choose a TTL longer than the expected processing time.
- Implement lock renewal for long-running tasks.
- Use fencing tokens for critical resources.
- Redis cannot automatically prevent unsafe operations after TTL expiry.

Scenario 35

A team introduces Redlock and assumes the protected database can no longer experience conflicting writes. What assumption is missing?

Answer and Explanation

1. Redlock reduces the chance of split-brain, but it does not guarantee strong consistency for external resources.
2. It assumes the system can tolerate small clock drift and network delays, which may not always be true.
3. The protected resource (e.g. database) must enforce correctness, for example using fencing tokens, version checks or transactional constraints.
4. Without fencing tokens or resource-side validation, conflicting writes can still occur.

Redlock Concept (Simplified)



Key Takeaways

- Redlock improves availability during failures.
- It does not guarantee external resource correctness.
- Use fencing tokens (monotonic version numbers) to prevent stale owners from writing.
- Combine Redis locks with database-level constraints or idempotent operations.

Production Focus

SET NX PX and lock tokens | Lease expiration and renewal | Fencing tokens and correctness | Redlock and failure modes | External resource validation | Availability vs consistency

VERIQTA

14. The Redis Queue That Lost Work Without Reporting an Error

Scenario 36

A worker removes a job from a Redis List and crashes before processing it. How can the queue design prevent silent loss?

Answer and Explanation

1. Using a simple LPOP removes the job permanently from the list. If the worker crashes before processing, the job is lost.
2. Use a reliable queue pattern with a processing list. Move the job to a processing list using BRPOPLPUSH so it is not lost if the worker crashes.
3. After successful processing, remove the job from the processing list.
4. Implement a recovery mechanism to requeue stuck jobs from the processing list (e.g. based on a timeout).
5. Consider using Redis Streams for better reliability, visibility, and pending message tracking.

Useful Commands

Reliable queue with lists

```
BRPOPLPUSH queue processing 0
LREM processing 1 <job-id>
LRANGE processing 0 -1
```

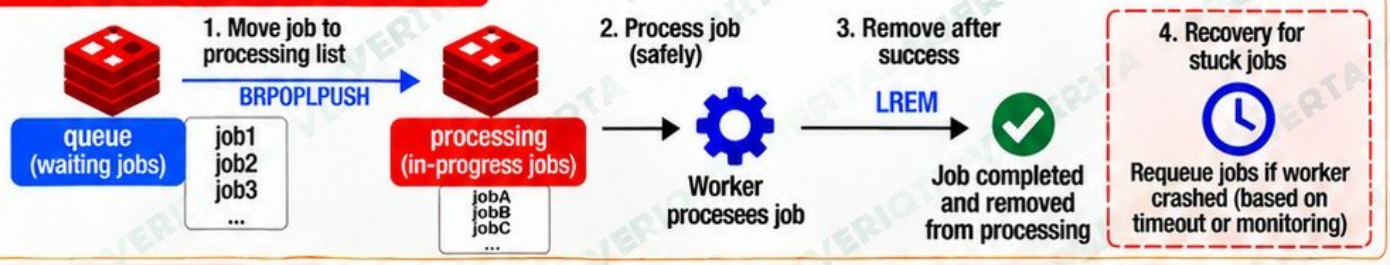
Move stuck jobs back to queue (e.g. older than 5 minutes)

```
LMOVE processing queue LEFT RIGHT
# or use a timestamp in the job and
a background worker to requeue
```

Key Takeaways

- LPOP removes the job permanently and can cause silent loss if the worker crashes.
- Use BRPOPLPUSH/LMOVE to implement a reliable queue pattern.
- Keep jobs in a processing list until successfully completed.
- Implement a recovery process for stuck jobs.
- Redis Streams provide better reliability and observability for production systems.

Reliable Queue Pattern with Lists



Scenario 37

Redis Streams consumers stop making progress while pending entries continue increasing. What evidence would you inspect?

Answer and Explanation

1. Check pending entries using **XPENDING** to see the number of unacknowledged messages.
2. Identify which consumer has the pending messages and how long they have been idle.
3. The consumer may have crashed, stalled, or failed to acknowledge messages (XACK).
4. Inspect consumer group information and consumer liveness (XINFO GROUPS, XINFO CONSUMERS).
5. Look at application logs, processing time, and infrastructure issues (CPU, memory, network).

Useful Commands

```
XINFO GROUPS <stream>
XINFO CONSUMERS <stream> <group>
XPENDING <stream> <group>
XPENDING <stream> <group> - + 10
XCLAIM <stream> <group> <consumer>
        <min-idle-time> <message-id>
XAUTOCLAIM <stream> <group>
  <consumer> <min-idle-time> 0 COUNT 10
```

Key Takeaways

- Pending entries increase when messages are not acknowledged.
- Use XPENDING and XINFO to identify stuck consumers.
- Consumers may have crashed, be slow, or have bugs.
- Use XAUTOCLAIM or XCLAIM to reclaim idle messages.
- Monitor consumer health and processing latency.

Scenario 38

A Redis Streams message is processed twice after consumer recovery. How should the application handle this correctly?

Answer and Explanation

1. Redis Streams provide at-least-once delivery, so duplicates can occur (e.g. after a consumer crash and message reclaim).
2. The application must be **idempotent** so that processing the same message multiple times does not cause incorrect results.
3. Use a unique message ID or business key to check for duplicates (e.g. store in Redis or database).
4. Acknowledge the message only after successful processing.
5. Handle retries and duplicates gracefully in the application logic.

Example Idempotency Pattern

```
# Check if message already processed
SETNX processed:<message-id> 1
EXPIRE processed:<message-id> 86400

# Process the message only if not already done
if SETNX succeeds:
    process_message()
    XACK <stream> <group> <message-id>
else:
    # Already processed, just acknowledge
    XACK <stream> <group> <message-id>
```

Key Takeaways

- Redis Streams can deliver the same message more than once.
- Design consumers to be idempotent.
- Use a unique message ID or business key to detect duplicates.
- Acknowledge messages after successful processing.
- At-least-once delivery is the default behavior, so duplicates are normal and must be handled.

Production Focus

Reliable queue patterns | XREADGROUP and consumer groups | XPENDING and XAUTOCLAIM | Acknowledgments and reclaim behavior | Idempotency and duplicates | At-least-once processing

VERIQTA

15. The Cache Is Correct Individually but Wrong as a System

Scenario 39

A database update succeeds, but Redis continues returning an old value. How do you investigate cache invalidation races?

Answer and Explanation

1. Check the application's cache invalidation flow and ordering (update DB → invalidate cache).
2. Look for multiple application instances where invalidations may be delayed, lost, or executed before the database commit.
3. Verify if the application reads from cache before the invalidation has propagated.
4. Check for asynchronous invalidation via a message queue and inspect for delivery delays or failures.
5. Review logs, timestamps, and request traces to identify the race condition and the exact sequence of events.

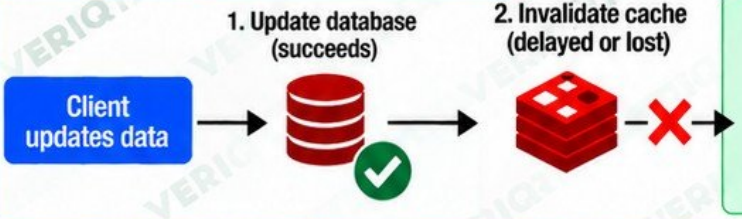
Useful Commands

```
redis-cli  GET user:123
redis-cli  TTL user:123
redis-cli  MONITOR
redis-cli  --latency-history
redis-cli  INFO stats
redis-cli  SLOWLOG GET 10
# check key version if used
redis-cli  GET user:123:version
# check last update time metadata
redis-cli  HGETALL user:123:meta
```

Key Takeaways

- Cache invalidation races happen when cache and database updates are not properly ordered or synchronized.
- Invalidation can be lost, delayed, or executed before the database commit.
- Use atomic update and invalidate patterns or versioned values.
- Consider write-through or event-driven invalidation.
- Monitor and trace request flow across all application instances.
- Ensure reliable message queues if using asynchronous invalidation.

Cache Invalidation Race (Example)



Possible Causes

- Invalidation happens before DB commit
- Invalidation message is lost or delayed
- Multiple app instances with stale cache
- Asynchronous processing delays
- Network issues or queue failures
- Missing version check or conditional update

Scenario 40

Two concurrent application requests overwrite a newer cached value with an older one. What ordering controls could prevent this?

Answer and Explanation

1. Two concurrent requests read the same old value, both update the database, and then write to Redis out of order.
2. The slower request may overwrite the newer cached value with an older one.
3. Use versioned values, timestamps, or a compare-and-set (CAS) pattern to prevent stale writes.
4. Store a version or last-updated timestamp with the cached value and only update if the new version is newer.
5. Alternatively, use a write-through cache, a distributed lock, or an atomic Lua script to ensure ordering.

Example: Versioned Cache Update

Client A (newer update)

```
SET user:123 "v2:data"
SET user:123:version 2
```

Client B (older update, arrives later)

```
# Only update if version is newer
if tonumber(redis.call('GET',
   KEYS[1])) < tonumber(ARGV[1]) then
    redis.call('SET', KEYS[2], ARGV[2])
    redis.call('SET', KEYS[1], ARGV[1])
    return 1
else
    return 0
end
-- KEYS[1] = user:123:version
-- KEYS[2] = user:123
-- ARGV[1] = new_version
-- ARGV[2] = new_value
```

Key Takeaways

- Concurrent requests can overwrite newer data with older values without proper ordering controls.
- Use version numbers, timestamps, or CAS to ensure only newer values are written.
- Consider write-through caching for stronger consistency.
- Distributed locks can prevent concurrent writes but may reduce performance.
- Define clear consistency boundaries and understand that caches are eventually consistent.

Production Focus

Cache-aside races | Write-through vs write-behind | Versioned values | Compare-and-set patterns | Invalidation ordering | Consistency boundaries

VERIQTA

16. Redis Was Compromised Without an Application Vulnerability

Scenario 41

A Redis instance is accessible outside its intended network boundary. What immediate containment and investigation steps should follow?

Answer and Explanation

1. Immediately restrict network access (block public access, update firewall/security group rules, allow only trusted IPs).
2. Check for unauthorized data access, configuration changes, or suspicious commands in Redis logs and audit sources.
3. Rotate all credentials (ACL users and passwords). and invalidate exposed secrets.
4. Investigate how the instance became exposed (misconfigured security group, open port 6379, incorrect Kubernetes/VM settings, etc).
5. Review system and network logs to identify attacker activity, persistence, and data exfiltration.

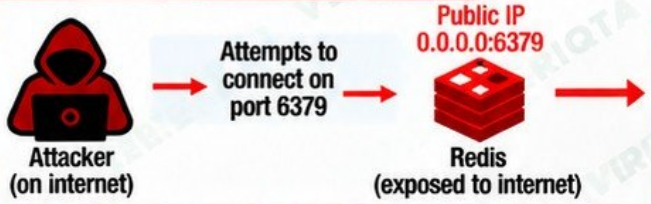
Useful Commands

```
# check if Redis is exposed
sudo nmap -p 6379 <server-ip>
# check Redis configuration
redis-cli CONFIG GET bind
redis-cli CONFIG GET protected-mode
# check connected clients
redis-cli CLIENT LIST
# check logs for suspicious commands
tail -n 100 /var/log/redis/redis.log
# view ACL users
redis-cli ACL LIST
# block access (example UFW)
sudo ufw deny 6379
# check security group (cloud CLI)
aws ec2 describe-security-groups --group-ids <sg-id>
```

Key Takeaways

- Never expose Redis to the internet.
- Use firewall rules, security groups, and private networks.
- Enable protected-mode and bind to trusted interfaces.
- Monitor logs and audit events regularly.
- Rotate credentials immediately after exposure.
- Investigate root cause and close configuration gaps.
- Use intrusion detection and network monitoring to detect unexpected access.

Example: Exposed Redis Instance



Possible impact

- Read or modify data
- Flush all data (FLUSHALL)
- Create backdoors
- Use Redis for further attacks
- Exfiltrate sensitive information

Correct Configuration

- Bind to private IP only
- Use firewall / security group
- Enable ACL and strong password
- Enable TLS for encryption
- Monitor and log access

Scenario 42

Redis ACLs are configured, but an application identity can still perform dangerous administrative operations. How would you redesign authorization?

Answer and Explanation

1. Review the application's ACL user permissions using ACL LIST and ACL GETUSER.
2. Remove unnecessary command categories and restrict to the minimum required commands.
3. Use separate ACL users for different workloads (app, admin, monitoring).
4. Deny dangerous commands like CONFIG, FLUSHALL, SHUTDOWN, ACL, and KEYS.
5. Regularly review and test ACL policies.

Example ACL Configuration

```
# create a restricted user for the application
ACL SETUSER appuser on >strongpassword
~app:* +@read +@write -@admin -@dangerous
# example admin user (restricted to specific IP)
ACL SETUSER admin on >adminpassword
~* +@all
# check user permissions
ACL GETUSER appuser
ACL LIST
```

Key Takeaways

- Follow least-privilege principles.
- Use ACL categories to allow only required commands.
- Separate application and administrative identities.
- Remove access to dangerous commands.
- Regularly audit and test ACL configurations.
- Use strong passwords and rotate them periodically.

Scenario 43

Redis traffic is encrypted, but credentials remain exposed through deployment configuration and operational tooling. What security controls are missing?

Answer and Explanation

1. Identify where credentials are stored (env vars, configuration files, CI/CD pipelines, logs, tooling).
2. Remove hardcoded secrets and use a secure secret management solution (e.g, AWS Secrets Manager, HashiCorp Vault, or Kubernetes Secrets).
3. Implement automatic credential rotation.
4. Restrict access to secrets to only authorized workloads and operators.
5. Enable audit logging and monitor for secret access.

Examples of Secure Secret Usage

```
# use environment variables (in Kubernetes)
env:
  - name: REDIS_PASSWORD
    valueFrom:
      secretKeyRef:
        name: redis-secret
        key: password
# fetch secret from AWS Secrets Manager
aws secretsmanager get-secret-value \
 --secret-id prod/redis --query SecretString
```

Key Takeaways

- Encryption in transit (TLS) is not enough if credentials are exposed.
- Do not store secrets in code, images, or plain text configuration files.
- Use a secure secret management system.
- Implement credential rotation.
- Restrict access to secrets and monitor their usage.
- Enable audit logging for configuration and secret access.

Production Focus

Network isolation | TLS encryption | ACL categories | Credential rotation | Audit evidence | Least-privilege workload identities

VERIQTA

17. Redis Monitoring Says Healthy While Users Report an Outage

Scenario 44

Redis responds to PING, but application requests are timing out. Why is the health check insufficient?

Answer and Explanation

1. PING only checks basic connectivity and that the server process is running.
2. PING does not validate command execution, latency, queue length, blocked clients, or application paths.
3. Redis can respond to PING while specific commands are slow, blocked, or timing out.
4. Network issues, connection exhaustion, or client timeouts can cause application failures even when PING succeeds.
5. Use application-level health checks that perform real read/write operations and measure latency from the application's perspective.

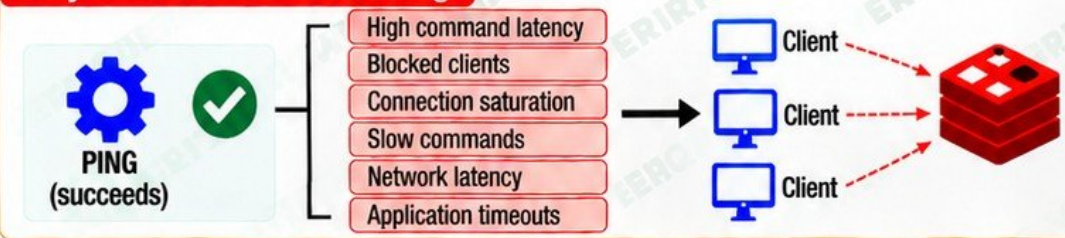
Useful Commands

```
redis-cli  PING
redis-cli  INFO stats
redis-cli  INFO clients
redis-cli  INFO commandstats
redis-cli  LATENCY DOCTOR
redis-cli  SLOWLOG GET 10
redis-cli  CLIENT LIST
redis-cli  INFO replication
# measure latency from application
redis-cli --latency -h <host> -p 6379
# check blocked clients
redis-cli  CLIENT LIST | grep b
# check slow commands
redis-cli  SLOWLOG GET 10
```

Key Takeaways

- PING is not a full health check.
- Validate real application commands (e.g. GET/SET).
- Monitor command latency, error rates, blocked clients, and connection count.
- Check network latency between application and Redis.
- Use synthetic transactions and application-level SLIs.
- Configure alerts on latency, errors, and saturation, not just process availability.

Why PING Can Be Misleading



Result

- Application requests time out
- Users see errors
- Monitoring still shows healthy (PING is OK)

Scenario 45

Redis CPU, memory, and uptime look normal, yet customer-facing latency has doubled. Which additional signals would reveal the failure?

Answer and Explanation

1. Check command latency histograms to identify slow operations (e.g. GET, SET, Lua scripts).
2. Monitor error rates (timeouts, connection errors, MOVED/ASK, OOM).
3. Review blocked clients and long-running commands.
4. Check connection saturation (connected_clients, maxclients, rejected_connections).
5. Analyze hit rate, evictions, replication lag, network latency, and application-level SLIs (e.g. p95/p99 latency).

Key Metrics to Monitor

Metric	What it Reveals
Command latency (p50/p95/p99)	Slow commands or contention
Error rate	Timeouts, failures
Blocked clients	Long-running operations
Connected clients	Saturation risk
Rejected connections	Client connection issues
Hit rate (keyspace_hits/misses)	Cache effectiveness
Evicted keys	Memory pressure
Replication lag	Stale reads, sync issues
Network latency	Network or infrastructure problems
Application SLIs	Real user impact

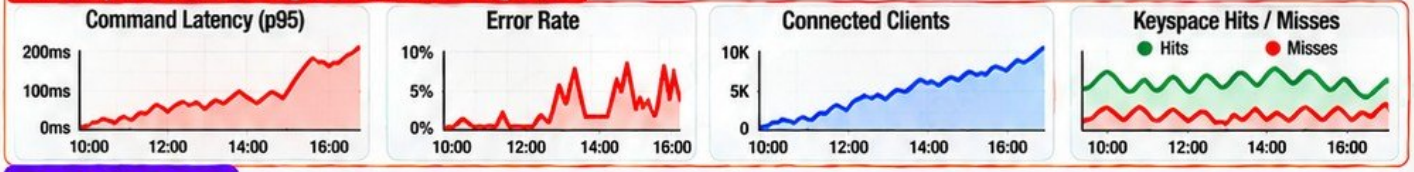
Useful Commands

```
redis-cli  INFO stats
redis-cli  INFO clients
redis-cli  INFO memory
redis-cli  INFO keyspace
redis-cli  LATENCY HISTOGRAM
redis-cli  LATENCY DOCTOR
redis-cli  SLOWLOG GET 10
redis-cli  CLIENT LIST
redis-cli  INFO replication
```

Key Takeaways

- Monitor latency, errors, and saturation.
- Track hit rate, evictions, and replication lag.
- Use command latency histograms.
- Monitor application-level SLIs (p95/p99).
- Set alerts on user impact, not just CPU or memory.

Example Monitoring Dashboard Signals



Production Focus

Command latency histograms | Error rates | Blocked clients | Connection saturation | Hit rate and evictions | Replication lag | Network latency | Application SLIs

VERIQTA

18. The Redis Upgrade That Changed Production Behavior

Scenario 46

A Redis upgrade completes successfully, but application latency and memory consumption change. How would you isolate version-related behavior?

Answer and Explanation

1. Confirm the exact version change (e.g. 6.2 → 7.0) and review the release notes for known behavior changes.
2. Compare metrics before and after the upgrade (latency, memory, CPU, connections, evictions).
3. Run controlled tests with the new version (e.g. same workload, same data set).
4. Check configuration differences (e.g. maxmemory, eviction policy, AOF/RDB settings).
5. Isolate by testing a single node or canary instance before full rollout.

Useful Commands

```
# check Redis version
redis-cli  INFO server

# compare configuration
redis-cli  CONFIG  GET *

# monitor key metrics
redis-cli  INFO stats
redis-cli  INFO memory
redis-cli  INFO persistence

# run a simple latency test
redis-benchmark -n 10000 -c 50

# check for slow commands
redis-cli  SLOWLOG GET 10
```

Key Takeaways

- New versions can change memory usage, latency, and internal behavior.
- Always review release notes before upgrading.
- Use canary or staged rollouts to detect issues early.
- Compare metrics and performance with the old version.
- Keep configuration consistent across environments.

Common Version-Related Changes

- New features and defaults
- Memory behavior changes
- Performance improvements/ regressions
- Configuration changes
- Bug fixes or new bugs

Production Tip

- Always test upgrades with real workloads and monitor closely during and after the rollout.

Scenario 47

A team plans to roll back Redis after a failed upgrade. Why might restoring older binaries against newer data files be unsafe?

Answer and Explanation

1. Newer data files may use a format not supported by older versions (RDB/AOF format changes).
2. The older version may fail to load the data, causing startup errors or data loss.
3. Features and commands introduced in the newer version may leave incompatible data.
4. Even if it starts, behavior may be inconsistent or unpredictable.
5. Always restore from a verified backup taken before the upgrade.

Useful Commands

```
# check data file format
redis-check-rdb dump.rdb

# check AOF compatibility
head -n 100 appendonly.aof

# verify version
redis-server --version

# test restore (on staging)
redis-server /path/to/data

# check logs for errors
tail -f  /var/log/redis/redis.log
```

Key Takeaways

- Backward compatibility is not guaranteed between versions.
- Newer data files may not work with older binaries.
- Always use backups, not the live data files, for rollback.
- Verify the restored instance before switching traffic.
- Plan rollback steps before upgrading.

Safe Rollback Process

1. Take backup before upgrade
2. Upgrade and test thoroughly
3. If needed, restore from backup (not data files)
4. Verify the restored instance
5. Switch traffic only after validation

Production Focus

- Version compatibility
- Release notes
- Persistence-format compatibility
- Staged rollouts
- Canary workloads
- Backups
- Rollback planning
- Restoration verification

VERIQTA

19. Should Redis Be a Cache, a Database, or a Messaging System?

Scenario 48

A company wants to use Redis as the authoritative store for payment transactions because it is fast. How would you evaluate the proposal?

Answer and Explanation

1. Understand the requirements: durability, transaction guarantees, consistency, compliance, and recovery objectives.
2. Evaluate Redis capabilities: strong performance but limited durability and transactional guarantees for financial data.
3. Compare with database options (e.g. PostgreSQL, MySQL) that provide ACID, auditing, and proven recovery mechanisms.
4. Consider using Redis only as a cache or auxiliary store, not as the sole authoritative source for payment transactions.
5. Recommend a design with a real database as the source of truth and Redis for performance (caching, queues, sessions, etc).

Key Evaluation Points

- Redis is in-memory with optional persistence (RDB/AOF) but can still lose recent data.
- It does not provide full ACID transactions like traditional databases.
- Financial transactions require strong durability, auditing, and compliance (e.g. PCI DSS).
- Redis is excellent for speed but not the best choice as the sole authoritative store for payments.
- Use Redis as a cache or for supporting components, with a durable database as the system of record.

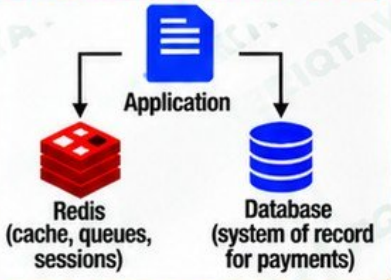
Key Takeaways

- Redis is not recommended as the primary database for payment transactions.
- Use a database with ACID guarantees as the source of truth.
- Redis can be used for caching, queues, or session storage to improve performance.
- Consider compliance requirements (e.g. PCI DSS, audit logging).
- Design for durability, consistency, and recoverability.

Comparison: Redis vs Traditional Database for Payments

Feature	Redis	Traditional Database (e.g. PostgreSQL)
Data model	In-memory key-value	Relational / ACID
Transaction guarantees	Limited (MULTI/EXEC, no full ACID)	Full ACID transactions
Durability	RDB/AOF (can lose recent data)	Strong durability and crash recovery
Consistency	Eventual in some setups	Strong consistency
Auditing / compliance	Limited built-in support	Mature auditing and compliance features
Best use case	Cache, sessions, queues	Authoritative store for critical data (e.g. payments)

Recommended Architecture



Scenario 49

A team uses Redis for caching, distributed locks, sessions, and job queues on one shared deployment. What hidden failure domains does this introduce?

Answer and Explanation

1. Multiple workloads share the same resources (CPU, memory, network, persistence).
2. A spike in one workload (e.g. large cache usage) can impact others (e.g. job queues, sessions).
3. Failure in one component can affect all services, increasing the blast radius.
4. Operational changes (upgrades, configuration changes) become riskier.
5. Isolation by workload (separate clusters or instances) reduces risk and improves stability.

Hidden Failure Domains

- Memory exhaustion from cache data can evict session or queue data.
- Long-running commands can block all workloads.
- Network or connection limits can affect multiple services simultaneously.
- A misconfiguration or upgrade failure impacts every workload.
- Difficult troubleshooting due to mixed workloads and metrics.

Key Takeaways

- Isolate different workloads (cache, queues, sessions, locks) into separate Redis instances or clusters.
- Define resource limits (memory, maxclients) per workload.
- Monitor each workload independently,
- Reduce blast radius with clear operational ownership.
- Plan capacity and recovery per workload based on its requirements.

Production Focus

Durability requirements | Transaction guarantees | Workload isolation | Operational ownership | Blast radius | Cost and ROIs | Recovery objectives | Architectural fitness

VERIQTA

20. Final Production Incident: Redis Is Available, but the Business Is Down

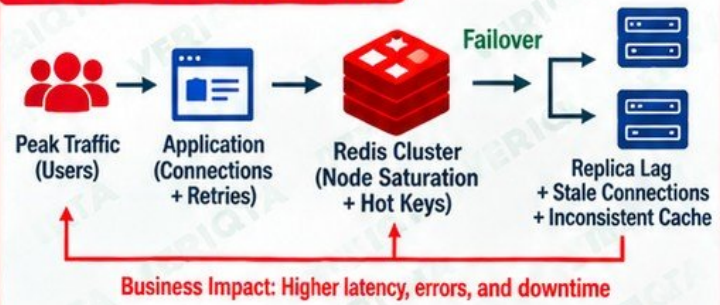
Scenario 50

During peak traffic, a Redis Cluster node becomes saturated. Hot-key traffic, client retries, connection growth, and replica lag compound the incident. A failover occurs, but stale application connections and inconsistent cache state continue affecting users.

Interview Question

You are the senior engineer leading the incident. What do you investigate, mitigate, verify, and change to prevent recurrence?

Production Incident Flow



The Answer Must Cover:

1. Define the customer impact and affected failure domains.
2. Correlate Redis latency, client errors, connection growth, hot-key traffic, and failover events.
3. Distinguish the initiating failure from secondary retry amplification.
4. Apply safe mitigations without causing a second outage.
5. Verify replication health, application recovery, and data correctness.
6. Establish long-term controls for workload isolation, backpressure, client resilience, and capacity planning.
7. Defend the architecture changes in a Staff Engineer-level review.

Step 1: Investigate

- Confirm customer impact (users, revenue, regions).
- Check Redis latency, client errors, connection growth, hot-key traffic.
- Review failover logs and replica lag.
- Find the initiating failure vs retry amplification.
- Look for stale connections and inconsistent cache state.

Step 2: Mitigate

- Throttle or block hot-key traffic.
- Reduce client retries and add backoff.
- Scale the affected node or rebalance.
- Close stale connections and refresh client pools.
- Avoid risky actions that could cause a second outage.

Step 3: Verify

- Replication health is stable and in sync.
- Application recovery is complete (without errors).
- Data correctness is validated.
- Latency and connection growth are normal.
- Monitor for at least 30–60 minutes.

Step 4: Prevent Recurrence

Workload isolation | Backpressure and rate limiting | Client resilience (retry + jitter) | Capacity planning | Monitoring and alerting | Better architecture and runbooks

Key Production Focus

Latency & error rates | Connection growth | Hot-key traffic | Replication lag | Failover events | Cache consistency | SLA / SLO objectives